



Written by L'Architech

INTRODUCTION

TorusNet is a sovereign directional architecture designed to establish secure communications without relying on traditional cryptographic mechanisms. Unlike classical protocols based on derivable mathematical structures, private keys, or certificates, TorusNet is built upon a living internal geometry composed of directional tori whose structure evolves with every key.

Security is no longer ensured through mathematical operations, but through:

- directional inversion of normal/ghost bits,
- dynamic mutation of internal IDs,
- Host–Client–Host synchronization,
- multi-tier orbital cascading,
- directional physics of the private torus,
- hardware-bound quadruple identity,
- and directional destruction at the end of the session.

These mechanisms render the private torus non-transmissible, non-derivable, non-clonable, and impossible to simulate—even by quantum or photonic systems.

1. The Three Fundamental Tori

The TorusNet architecture relies on three directional structures:

1. **The Public Torus** A directional reading space used for exponential displacements produced by the orbital cascade.
2. **The Authentication Torus** Transmitted only once at the beginning of the session; it serves as the initial directional anchor point.
3. **The Private Torus** Reconstructed locally, never transmitted, living, oscillating, self-modifying, and impossible to derive.

The Private Torus possesses a unique ID encoded within the fractal distribution of its internal IDs. This structural ID allows the Host to trigger directional renewal through a special key.

2. The Directional Key: 64 Bits, Including 30 Orbital Bits

Each directional key is 64 bits. It contains:

- type information (00, 01, 10, 11),
- directional identifiers,
- session parameters,
- and a 30-bit segment dedicated to directional displacements.

This 30-bit segment is not treated as a linear sequence: it is transformed into a circular ring of 30 positions, generating:

- 30 direct readings,
- 30 inverse readings,
- for a total of 60 orbital states,
- producing 1800 bits,
- divided into 180 directional blocks (N,E).

These displacements then read:

- 5400 bits from the Public Torus (tier 2),
- then 16200 bits,
- then 48600 bits,
- up to 145800 bits (tier 5).

The cascade is bijective, fully reversible, and allows reconstruction of either the private torus or the key from the other.

3. Host–Client–Host Directional Sequence

The transmission of a directional key follows exactly five steps, each ensuring perfect synchronization between the Host's Private Torus and the Client's. This sequence guarantees that no directional divergence is possible.

3.1. Host → Client: Initial Transmission of the Key

The Host sends the raw directional key to the Client. This key contains:

- the type (00, 01, 10, 11),
- the targeted case ID,
- the 30 fractal displacement bits,
- the directional information required for the traversal.

The Client receives the key but does not execute it yet.

3.2. Client → Host: Returning the Key for Validation

The Client sends back the exact same key to the Host.

The Host:

- compares the sent and received keys,
- verifies that no alteration occurred,
- confirms that the Client is the legitimate sender,
- registers the Client's IP as a trusted IP for the session.

If the key differs, the session is immediately destroyed.

3.3. Host → Client: Sending the Validated Copy

The Host sends a second copy of the key, this time validated.

The Client:

- compares the validated key with the one previously returned,
- confirms coherence,
- obtains authorization to execute the directional traversal.

From this moment, the key is considered authenticated.

3.4. Client → Host: Returning the Key After Execution

The Client executes the key within its Private Torus:

- reading 3-bit blocks from the Public Torus,
- selecting internal paths,
- performing normal/ghost inversions,
- mutating the case ID (first 16 bits),
- mutating the global ID (second 16 bits),
- completing the fractal progression.

Once execution is complete:

The Client returns the key to the Host to indicate that its torus has been updated. This return is the directional proof that the traversal was executed.

3.5. Host → Client: Final Confirmation and Synchronization

The Host executes the key within its own Private Torus:

- reading the same blocks,
- following the same internal paths,

- performing the same inversions,
- reconstructing the same IDs,
- updating its torus identically.

Once its torus is updated:

The Host returns the key to the Client to confirm reception AND confirm that its torus has been updated.

This step finalizes synchronization.

At the end of this step:

Both private tori are perfectly synchronized. No structural divergence is possible.

4. Directional Physics and Torus Life Cycle

Each case of the Private Torus contains:

- a directional ID,
- a normal bit,
- a ghost bit,
- four cardinal/diagonal directions,
- exponential NE displacements.

Before reading, the normal/ghost bits exist in directional superposition. Reading collapses the state, similar to a quantum spin.

The Private Torus:

- lives in RAM,
- is never transmitted,
- is encrypted at the end of the session,
- is reborn at each permutation,
- is structurally destroyed in case of anomaly.

5. Hardware-Bound Quadruple Identity

Reconnection requires exact correspondence of:

- MAC address,
- BIOS ID,
- CPU ID,
- internal Private Torus ID.

This quadruple identity makes the torus:

- non-transferable,
- non-clonable,
- unusable outside its original hardware,
- impossible to simulate or virtualize.

6. HTTPS and DNS: Temporary Mechanisms

TorusNet still uses:

- HTTPS for the first key,
- DNS as a preliminary filter.

These mechanisms are temporary and will be replaced by:

- a directional DNS,
- pre-session directional validation,
- native directional identity.

7. Disclosure Policy: One-Year Delay

An official delay of 12 months is granted to contacted institutions to:

- analyze the protocol,
- request information,
- propose collaboration.

Without response after one year, TorusNet and Mondus are released as open-source.

8. Mondus: The Post-Internet Network

Mondus is not an evolution of the Internet. It is a post-Internet directional network built upon:

- living tori,
- directional geometry,
- structural security,
- hardware-bound identity,
- non-transmissive communication.

Mondus represents the transition toward a future where networks no longer transmit data, but **directions**, and where geometry becomes the foundation of digital security.

Chapter 1 — Structure of Directional Tori

1.1. Overview

The TorusNet architecture is built upon three living directional tori, each possessing its own internal geometry and playing a critical role in the structural coherence of the session. These tori are not classical mathematical objects: they are closed directional spaces, non-derivable, non-transmissible, and interpretable only through the fractal, orbital, and directional mechanics of the protocol.

The three tori are:

1. **The Public Torus**
2. **The Authentication Torus**
3. **The Private Torus**

Each torus has a specific internal structure and a precise directional role within the session.

1.2. The Public Torus

1.2.1. Definition

The Public Torus is an exhaustive directional space containing all possible combinations of the 30 bits. It is no longer an arbitrary torus: it is a complete directional reservoir, covering the entirety of possible states for the 3-bit blocks used by the orbital cascade.

It contains no sensitive information, no private keys, and no derivable structure. Its role is strictly operational and universal: all clients use the same Public Torus.

1.2.2. Size and Internal Structure

The Public Torus contains:

$$2^{30} = 1,073,741,824 \text{ cases}$$

Each case contains exactly 30 bits, representing a unique combination among the 1,073,741,824 possible states.

These 30 bits are used for:

- orbital reading,
- fractal progression,
- internal coherence of the Private Torus,
- directional stability of the protocol.

Unlike the other tori, it contains:

- no normal/ghost pair,
- no internal path,
- no directional ID.

It is a universal directional reservoir, identical for all users.

1.2.3. Role in the Session

The Public Torus:

- provides the 30-bit blocks for exponential displacements,
- serves as a stable directional base for the orbital cascade,
- influences the reconstruction of the Private Torus,
- contains no exploitable data for an attacker.

1.3. The Authentication Torus

1.3.1. Definition

The Authentication Torus is transmitted only once at the beginning of the session. It establishes the initial coherence between the Host and the Client.

1.3.2. Size and Internal Structure

The Authentication Torus is a 1000×1000 grid, i.e., **1,000,000 cases**, each containing 3 bits:

- **Bit 0:** normal (0) or ghost (1)
- **Bits 1–2:** internal path (00, 01, 10, 11)

It also contains:

- initialization internal paths,
- coherent normal/ghost pairs,
- initial directional identifiers.

1.3.3. Role in the Session

The Authentication Torus:

- validates the first directional key,
- establishes initial coherence,
- initializes the global ID,
- prepares the reconstruction of the Private Torus.

Once this step is completed, it is no longer used.

1.4. The Private Torus

1.4.1. Definition

The Private Torus is the central element of TorusNet. It is reconstructed locally, never transmitted, and constitutes a directional space that is:

- living,
- oscillating,

- self-modifying,
- impossible to derive,
- impossible to simulate.

1.4.2. Size and Internal Structure

The Private Torus is a 1000×1000 grid, i.e., **1,000,000 cases**, each containing:

- a normal bit,
- a ghost bit,
- an internal path (00, 01, 10, 11),
- a directional ID (16 bits),
- four cardinal/diagonal directions,
- exponential NE displacements.

It also contains:

- a unique torus ID, encoded in the fractal distribution of internal IDs,
- a global ID, updated with each key.

Its structure evolves with each validated directional key through:

- normal/ghost inversion,
- ID mutation,
- fractal progression,
- Host–Client–Host synchronization.

1.4.3. Fractal and Orbital Reconstruction (Corrected)

Each directional key (64 bits) triggers a directional reconstruction based on the orbital cascade. The key contains a 30-bit displacement segment transformed into a circular ring enabling:

- 30 direct readings,
- 30 inverse readings,
- 60 orbital states,
- producing 1800 bits (tier 1).

These 1800 bits are divided into 180 directional blocks (N,E), each defining an exponential displacement in the Public Torus.

Reconstruction then proceeds through successive tiers, each amplifying the reading:

Tier	Bits Generated	Displacements	Description
1	1800 bits	180	Circular orbit (60 readings $\times$ 30 bits)
2	5400 bits	540	Public torus reading $\times$ 30 bits
3	16200 bits	1620	Directional amplification

Tier	Bits Generated	Displacements	Description
4	48600 bits	4860	Fractal expansion
5	145800 bits	14580	Complete directional projection

The Public Torus is thus read 10 times in cascade, according to directional physics:

- 10 fixed levels,
- 10 successive readings,
- 10 directional collapses,
- ensuring reversibility and stability of the Private Torus.

Orbital and fractal reconstruction is:

- non-derivable,
- non-transmissible,
- non-reversible,
- entirely dependent on the internal state of the Private Torus,
- stabilized by normal/ghost inversion and ID mutation.

1.4.4. Role in the Session

The Private Torus:

- validates directional keys,
- generates internal IDs,
- ensures directional coherence,
- protects the session from any derivability,
- forms the foundation of structural sovereignty.

It is directionally destroyed at the end of the session or in case of severe anomaly.

1.5. Relationships Between the Three Tori

1.5.1. Public Torus → Private Torus

The Public Torus provides the 3-bit blocks required for the orbital cascade and fractal reconstruction of the Private Torus.

1.5.2. Authentication Torus → Private Torus

The Authentication Torus validates the first key and initializes the directional coherence of the Private Torus.

1.5.3. Private Torus → Session

The Private Torus determines:

- key validity,

- fractal progression,
- directional coherence,
- structural security.

1.6. Structural Properties of the Tori

The three tori share common properties:

- non-transmissible (except Public Torus + Authentication Torus),
- non-derivable,
- non-reversible,
- non-exploitable,
- geometric,
- directional,
- fractal,
- inviolable by design.

These properties form the foundation of TorusNet's post-quantum and post-photon security.

1.7. Summary of Chapter 1

Chapter 1 establishes the fundamental structure of TorusNet:

- an exhaustive Public Torus of 2^{30} cases $\times$ 30 bits,
- an Authentication Torus of 1000×1000 cases $\times$ 3 bits,
- a living, oscillating, self-modifying Private Torus,
- a non-derivable internal geometry,
- a fractal and orbital reconstruction mechanism,
- strict separation of roles,
- complete structural sovereignty.

These elements form the foundation upon which rest:

- directional keys,
- Host–Client–Host transmission,
- the Glass Fortress,
- synchronization,
- multi-torus architecture,
- directional security,
- and structural destruction.

Chapter 2 — Structure of Directional Keys

2.1. Overview

Directional keys are the fundamental unit of progression within the protocol. Each key is a 64-bit packet transmitted through a bidirectional directional sequence synchronized between the Host and the Client.

A directional key is not a cryptographic command: it serves solely as a directional vector, triggering the internal orbital and fractal mechanics that modify the Private Torus.

A directional key:

- contains no sensitive information,
- cannot be derived,
- cannot be reversed,
- cannot be exploited in isolation,
- only has meaning within the context of the Private Torus and the directional cascade.

2.2. Official Breakdown of the 64 Bits

The internal structure of a directional key is strictly defined. It is not a cryptographic format, but a directional structure enabling:

- navigation within the Public Torus,
- reconstruction of the Private Torus,
- definition of the key type,
- internal directional validation,
- ID mutation,
- Host–Client–Host synchronization.

2.2.1. 64-Bit Breakdown Table

Segment	Bits	Size	Description
Starting Point	0–7	8 bits	4 bits for N, 4 bits for E → exponential displacement N^E
Ending Point	8–15	8 bits	Same logic: N^E
Key Type	16–17	2 bits	00 = reconstruction; 11 = key reconstruction; 01 = followed key; 10 = stop
Public Torus Navigation	18–47	30 bits	Circular orbital segment (ring)
Target Case ID	48–63	16 bits	Internal identifier of the targeted Private Torus case

2.3. Interpretation of the Segments

2.3.1. Starting Point (8 bits)

The 8 bits are divided into:

- 4 bits for an integer **N**,
- 4 bits for an exponent **E**,

forming an exponential displacement:

$$\Delta = N^E$$

This displacement determines the first case to read in the Public Torus.

2.3.2. Ending Point (8 bits)

Same structure:

- 4 bits for **N**,
- 4 bits for **E**,

forming:

$$\Delta = N^E$$

This displacement determines the last case to read in the Public Torus.

Together, they define a directional window.

2.3.3. Key Type (2 bits)

Bits	Type	Description
00	Reconstruction Key	Directional reading to reconstruct a file
11	Key Reconstruction Key	Used to reconstruct other keys
01	Followed Key	Another key immediately follows
10	Stop Key	Ends the sequence

These types define the directional logic of the session, not cryptographic commands.

2.3.4. Public Torus Navigation (30 bits)

The 30 bits form the orbital segment of the key.

They are transformed into:

- a circular ring of 30 positions,
- 30 direct readings,
- 30 inverse readings,
- for a total of 60 orbital states,
- producing 1800 bits (tier 1).

These 1800 bits are divided into 180 directional blocks (N,E), each defining an exponential displacement in the Public Torus.

Each displacement reads a 3-bit block from the Public Torus.

The 30 bits are never interpreted as commands: they are a geometric orbital structure.

2.3.5. Target Case ID (16 bits)

The 16 bits represent the internal identifier of the Private Torus case targeted by the key.

This ID:

- validates fractal coherence,
- ensures the targeted case is correct,
- prevents directional falsification,
- participates in directional mutation of the starting ID (Update 1).

It is a geometric identifier, not a cryptographic one.

2.4. Directional Role of Keys

2.4.1. Triggering the Orbital Cascade

Each key triggers:

- generation of the 30-bit ring,
- 60 orbital readings,
- 180 directional blocks (N,E),
- reading of the Public Torus across 10 successive tiers,
- normal/ghost inversion,
- ID mutation,
- updating of the Private Torus.

2.4.2. Reconstruction of the Private Torus

Directional keys:

- reconstruct the Private Torus,
- update normal/ghost pairs,
- mutate internal IDs,
- stabilize the global ID,
- validate internal paths,
- synchronize Host and Client.

2.4.3. Absence of Derivability

A directional key:

- cannot be predicted,

- cannot be reconstructed,
- cannot be reversed,
- cannot be analyzed in isolation,
- only has meaning within the context of the Private Torus.

2.5. Directional Transmission

Transmission now follows six steps (Update 1), ensuring perfect synchronization:

1. **Host** → **Client**: sending the key
2. **Client** → **Host**: returning the key
3. **Host** → **Client**: sending the validated copy
4. **Client**: executing the directional traversal
5. **Client** → **Host**: confirmation
6. **Host**: executing the same traversal

Both private tori evolve symmetrically.

2.6. Key Loss Management

If no response is received after 5 seconds:

- the Host generates a new key,
- with a different starting point,
- but reconstructing the same directional file.

The protocol never retransmits the same packet: it regenerates a coherent key.

2.7. Summary of Chapter 2

This chapter establishes:

- the official breakdown of the 64 bits,
- the internal orbital and directional logic,
- the six-step transmission sequence,
- directional regeneration in case of loss,
- impossibility of derivation,
- the role of tiers in reconstructing the Private Torus.

Directional keys are the engine of orbital and fractal progression.

Chapter 3 — Fractal Mechanics and 3-Bit Blocks

3.1. Overview

Fractal mechanics form the operational core of the protocol. They are responsible for:

- reading 3-bit blocks in the Public Torus,
- internal directional progression,
- reconstruction of the Private Torus,
- creation and mutation of internal identifiers,
- validation of internal paths,
- structural coherence of the session.

Each directional key triggers an orbital cascade composed of several successive tiers that amplify directional reading and modify the Private Torus.

Fractal mechanics are:

- directional,
- non-linear,
- non-derivable,
- dependent on the internal state of the torus,
- stabilized by normal/ghost inversion and ID mutation.

3.2. The 3-Bit Blocks: Fundamental Structure

The 3-bit blocks are the smallest directional unit of the protocol. They appear:

- in the Public Torus (extracted from the 30 bits),
- in the Authentication Torus,
- in the Private Torus.

Each 3-bit block is interpreted according to the following geometric rule:

Bit	Meaning
-----	---------

Bit 0	Normal (0) or Ghost (1)
-------	-------------------------

Bits 1–2	Internal path (00, 01, 10, 11)
----------	--------------------------------

This rule is inviolable. The 3-bit blocks do not encode instructions: they define internal direction only.

3.3. Interpretation of the Normal/Ghost Bit

The first bit of the block determines the nature of the displacement, but in the modern protocol, it has an essential property:

✓ **The normal bit and the ghost bit are inverted simultaneously for every validated key.**

This simultaneous inversion is a structural rule introduced in directional mechanics (Update 1). It transforms the Private Torus into a living, oscillating, anti-predictive space.

3.3.1. Normal Bit (0)

A normal bit indicates:

- a coherent displacement,
- stable progression,
- direct reading of the internal path.

During key validation:

- **normal = 0 → 1**
- this inversion modifies the internal geometry of the torus,
- and contributes to directional mutation of the case.

3.3.2. Ghost Bit (1)

A ghost bit indicates:

- an inverted displacement,
- indirect progression,
- reversed reading of the internal path.

During key validation:

- **ghost = 1 → 0**
- the simultaneous inversion creates directional oscillation,
- preventing any derivability or simulation of the torus.

3.3.3. Multiple Inversion

If the traversal passes through a case again:

- inversion occurs again,
- creating multiple directional oscillations,
- making the torus even more unpredictable.

3.4. The Four Internal Paths

Bits 1 and 2 define one of four internal paths:

Bits	Path	Description
00	Path A	Linear displacement
01	Path B	Diagonal displacement

Bits	Path	Description
10	Path C	Exponential displacement
11	Path D	Inverted displacement

These paths are not commands: they define the internal geometry of the torus.

3.5. Fractal Reading in the Public Torus

The 30 orbital bits of the directional key are transformed into:

- 30 direct readings,
- 30 inverse readings,
- for a total of 60 orbital states,
- producing 1800 bits (tier 1).

These 1800 bits are divided into 180 directional blocks (N,E). Each block (N,E) defines an exponential displacement in the Public Torus.

Each displacement reads a 3-bit block in the Public Torus.

The cascade then continues across 10 successive tiers, each amplifying the reading:

Tier	Bits Generated	Displacements
1	1800 bits	180
2	5400 bits	540
3	16200 bits	1620
4	48600 bits	4860
5	145800 bits	14580
6–10	Successive amplifications	Directional expansion

Fractal reading is:

- directional,
- non-linear,
- non-derivable,
- dependent on the internal state of the torus.

3.6. Internal Fractal Progression

Fractal progression follows three fundamental rules:

3.6.1. Coherence Rule

A displacement is valid if:

- the normal/ghost bit is coherent with the state of the case,
- the internal path is compatible with the previous progression.

3.6.2. Continuity Rule

Progression must:

- remain within the boundaries of the torus,
- follow internal paths,
- respect ghost inversions.

3.6.3. Expansion Rule

Exponential displacements (Path C):

- increase progression speed,
- allow rapid traversal of the torus,
- create and mutate internal IDs.

3.7. Directional Mutation of the Starting Case ID (Update 1)

When a key traverses the Private Torus:

✓ **The ID of the starting case automatically mutates into a new directional ID.**

This mutation:

- depends on the first bits of the key,
- modifies the directional identity of the case,
- changes the internal geometry of the torus,
- makes IDs dependent on the history of keys,
- prevents any derivability or external reconstruction.

ID mutation is a central mechanism of the Private Torus.

3.8. Special Case: Directional Destruction

Directional destruction occurs:

- at the end of the session,
- in case of severe anomaly,
- when a stop key (type 10) is used.

It consists of:

- rendering normal/ghost pairs incoherent,
- invalidating internal paths,
- erasing internal IDs,
- erasing the global ID,
- collapsing directional geometry.

Destruction is not random: it is structurally incoherent, preventing any post-mortem analysis—even by quantum or photonic systems.

3.9. Summary of Chapter 3

This chapter establishes:

- the structure of 3-bit blocks,
- the normal/ghost logic and their simultaneous inversion,
- the four internal paths,
- orbital reading in the Public Torus,
- the multi-tier directional cascade,
- directional mutation of internal IDs,
- directional progression,
- directional destruction.

Fractal mechanics are the internal engine of the Private Torus. They ensure progression that is:

- non-derivable,
- non-reversible,
- impossible to simulate,
- inviolable by design.

Chapter 4 — The Glass Fortress (Directional Firewall)

4.1. Overview

The Glass Fortress is the structural directional firewall of the protocol. Unlike traditional firewalls based on rules, signatures, ports, or heuristics, the Glass Fortress relies on a living internal geometry embedded within:

- the Public Torus,
- the Authentication Torus,
- the Private Torus,
- fractal mechanics,
- orbital logic,
- the Host → Client → Host directional progression.

It does not filter packets using classical network criteria. Instead, it validates the directional coherence of keys, displacements, IDs, internal paths, and normal/ghost inversions.

The Glass Fortress forms a structural barrier that cannot be bypassed, even by an attacker equipped with a quantum or photonic computer.

4.2. Structural Nature of the Glass Fortress

The Glass Fortress is:

- not software,
- not an external module,
- not a network component.

It is an internal structure integrated into:

- the Public Torus,
- the Authentication Torus,
- the Private Torus,
- fractal mechanics,
- the orbital cascade,
- directional logic.

It operates without:

- signatures,
- rules,
- TCP ports,
- MAC addresses,
- timestamps,
- sequence numbers,
- cryptography.

It relies solely on:

- fractal coherence,
- internal geometry,
- normal/ghost pairs,
- internal paths,
- directional IDs,
- the unique torus ID,
- ID mutation,
- Host → Client → Host progression.

4.3. Role of the Glass Fortress

The Glass Fortress ensures four essential functions:

1. **Directional validation of keys**
2. **Structural protection against intrusions**
3. **Monitoring of IPs and directional ports**
4. **Immediate destruction in case of anomaly**

These functions are not executed through software rules, but through the internal structure of the torus.

4.4. Directional Validation of Keys

The Glass Fortress validates each directional key according to:

- its type (00, 01, 10, 11),
- its starting point (N^E),
- its ending point (N^E),
- its 60 orbital readings,
- its 180 exponential displacements,
- the 3-bit blocks read in the Public Torus,
- the targeted case ID,
- internal fractal coherence,
- normal/ghost inversion,
- ID mutation of the starting case.

A key is **accepted** if:

- displacements are coherent,
- normal/ghost pairs are valid,
- internal paths are compatible,
- the targeted ID corresponds to a valid Private Torus case,
- ID mutation is legitimate,
- orbital progression is coherent.

A key is **rejected** if:

- a displacement is incoherent,
- an internal path is impossible,
- an ID corresponds to no case,
- a normal/ghost inversion is illegitimate,
- an ID mutation is incoherent.

Rejection is structural, not logical.

4.5. Structural Protection Against Intrusions

The Glass Fortress protects against:

- falsified packets,
- malformed keys,
- incoherent keys,
- derived keys,
- injected keys,
- repeated keys,
- hostile keys.

It does not compare packets to signatures. It verifies directional coherence.

An attacker cannot:

- derive a key,
- predict a key,
- reconstruct a key,
- falsify a key,
- inject a valid key.

Any hostile key is immediately rejected.

4.6. Monitoring of IPs and Directional Ports

The Glass Fortress monitors:

- the client's IP,
- the host's IP,
- directional ports,
- internal paths,
- global IDs,
- the unique torus ID.

4.6.1. Trusted IP

During the first validated key:

- the client's IP is registered as a trusted IP,
- any other IP is structurally rejected.

4.6.2. Directional Ports

Directional ports are not classical TCP ports. They are defined by:

- internal paths,
- normal/ghost pairs,
- internal IDs,
- directional geometry.

A directional port:

- cannot be scanned,
- does not exist in the classical network layer,
- cannot be falsified.

4.7. Detection of Directional Anomalies

The Glass Fortress detects the following anomalies:

- normal/ghost incoherence,
- internal path incoherence,
- ID incoherence,
- exponential incoherence,
- fractal progression incoherence,
- ID mutation incoherence,
- transmission incoherence (Host → Client → Host).

4.7.1. Transmission Incoherence

If the sequence:

1. Host → Client
2. Client → Host
3. Host → Client

is not respected, the key is rejected.

4.7.2. Fractal Incoherence

If a fractal action:

- reads an impossible block,
- reaches an incoherent case,
- creates an illegitimate ID,
- follows a forbidden path,
- produces an incoherent ID mutation,

the session is immediately destroyed.

4.8. Directional Destruction in Case of Anomaly

The Glass Fortress triggers directional destruction if:

- a key is incoherent,
- fractal progression is impossible,
- an ID is illegitimate,
- an ID mutation is incoherent,
- a directional port is violated,
- a hostile IP is detected.

Directional destruction consists of:

- rendering normal/ghost pairs incoherent,
- invalidating internal paths,
- erasing internal IDs,
- erasing the global ID,
- erasing the Private Torus.

This destruction is not random: it is structurally incoherent, preventing any post-mortem analysis.

4.9. Summary of Chapter 4

The Glass Fortress is a structural directional firewall. It:

- validates keys,
- protects against intrusions,
- monitors IPs and directional ports,
- detects fractal anomalies,
- destroys the session in case of incoherence,
- relies solely on directional geometry and fractal coherence.

It forms the fundamental structural barrier of the protocol.

Chapter 5 — Directional Handshake (Host → Client → Host → Client → Host Sequence)

5.1. Overview

The directional handshake is the fundamental mechanism that establishes a coherent session between a host and a client. Unlike classical protocols (TLS, SSH, IPSec) that rely on:

- private keys,

- certificates,
- signatures,
- mathematical derivations,
- cryptographic exchanges,

the directional handshake relies exclusively on:

- the internal geometry of the tori,
- fractal coherence,
- the orbital cascade,
- directional displacements,
- internal IDs,
- structural validation.

It contains no cryptography, no derivation, and no exploitable mathematical structure.

5.2. Objectives of the Handshake

The directional handshake ensures:

1. validation of the first directional key,
2. initial synchronization between host and client,
3. registration of the client's IP as a trusted IP,
4. creation of the session's global ID,
5. activation of the Glass Fortress,
6. initial reconstruction of the Private Torus,
7. establishment of the torus's unique ID,
8. complete directional synchronization through 5 transmissions.

These steps are structural, not cryptographic.

5.3. The Official Sequence: 5 Transmissions

The directional sequence consists of 5 transmissions, each with a precise structural role. It is mandatory, inviolable, and forms the foundation of directional synchronization.

Transmission 1 — Host → Client

Initial sending of the directional key

The host sends a 64-bit key composed of:

- 8 bits: starting point (N^E),
- 8 bits: ending point (N^E),

- 2 bits: key type,
- 30 bits: orbital ring,
- 16 bits: targeted case ID.

The client:

- receives the key,
- verifies its structure,
- verifies its type,
- verifies the targeted ID,
- verifies initial fractal coherence.

No orbital reading is executed yet.

Transmission 2 — Client → Host

Returning the key to confirm reception

The client returns the exact same key to the host.

The host:

- compares the sent and received keys,
- validates reception,
- registers the client's IP as a trusted IP,
- activates the Glass Fortress.

If the key differs → **immediate destruction**.

Transmission 3 — Host → Client

Final return of the key to authorize execution

The host sends a second copy of the key.

The client:

- compares the returned key with the received one,
- validates coherence,
- executes the directional traversal in its Private Torus,
- inverts normal/ghost bits,
- mutates the starting case ID,
- updates internal paths,
- completes the orbital cascade (tiers 1 to 10).

The client's Private Torus becomes living, oscillating, and modified.

Transmission 4 — Client → Host

Confirmation of traversal execution

The client returns the key after executing the traversal.

This return indicates to the host:

- that normal/ghost inversions were performed,
- that the starting case ID mutated,
- that internal paths were updated,
- that fractal progression is coherent,
- that the client's Private Torus is up to date.

The host can then:

- execute the same traversal,
- invert normal/ghost bits on its side,
- mutate the starting case ID,
- synchronize its Private Torus with the client's.

Transmission 5 — Host → Client

Final synchronization confirmation

The host returns the key one last time to confirm:

- that it executed the traversal,
- that it inverted normal/ghost bits,
- that it mutated the starting case ID,
- that its Private Torus is synchronized with the client's,
- that the session is directionally stable.

The client receives this confirmation, and the session becomes coherent, stable, and inviolable.

5.4. Why Five Transmissions?

5.4.1. Complete Bidirectional Validation

The host validates the client. The client validates the host. Then each validates the other's directional execution.

5.4.2. Protection Against Injection

An attacker cannot:

- intercept the key,
- modify it,

- return it,
- falsify it.

Any incoherence is structurally detected.

5.4.3. Protection Against Repetition

A repeated key:

- is detected,
- is rejected,
- destroys the session.

5.4.4. Protection Against Derivation

An attacker cannot derive:

- the starting point,
- the ending point,
- the 30 orbital bits,
- exponential displacements,
- the targeted case ID,
- ID mutation.

Directional structure is non-derivable.

5.4.5. Perfect Directional Synchronization

The 5 transmissions ensure that:

- both Private Tori executed the same traversal,
- normal/ghost inversions are symmetrical,
- ID mutations are identical,
- internal geometry is coherent,
- no divergence is possible.

5.5. Initial Synchronization

Once the key is validated:

- the Authentication Torus is used once,
- the Private Torus is initialized,
- the global ID is created,
- normal/ghost pairs are defined,
- internal paths are activated,

- the torus's unique ID is established,
- the Glass Fortress is armed.

This synchronization is fractal, not mathematical.

5.6. Initial Reconstruction of the Private Torus

The first key triggers:

- generation of the orbital ring,
- 60 orbital readings,
- 180 directional blocks (N,E),
- reading of 3-bit blocks in the Public Torus,
- mutation of the starting case ID,
- updating of internal paths,
- establishment of directional coherence.

The Private Torus becomes living, oscillating, and operational.

5.7. Error Handling During the Handshake

5.7.1. Incoherent Key

If the key:

- contains an invalid ID,
- contains an impossible starting point,
- contains an impossible ending point,
- contains an illegitimate type,

the session is destroyed.

5.7.2. Modified Key

If the key returned by the client differs:

- immediate destruction,
- erasure of the Private Torus,
- invalidation of the IP.

5.7.3. No Response

If the host receives no response after 5 seconds:

- the key is considered lost,
- a new key is generated,
- with a different starting point,

- but reconstructing the same directional file.

No retransmission of the same packet is ever performed.

5.8. Summary of Chapter 5

The directional handshake:

- relies on 5 transmissions,
- validates the first directional key,
- registers the client's IP as trusted,
- initializes the Private Torus,
- activates the Glass Fortress,
- creates the global ID,
- establishes the torus's unique ID,
- guarantees total directional coherence,
- protects against injection, repetition, derivation, and falsification.

It forms the foundation of structural directional security.

Chapter 6 — Directional Synchronization

6.1. Overview

Directional synchronization is the mechanism that ensures that:

- the Public Torus,
- the Authentication Torus,
- the Private Torus,
- directional keys,
- the orbital cascade,
- internal IDs,
- directional paths,
- the global ID,
- the torus's unique ID,
- and the Glass Fortress

remain perfectly aligned throughout the session.

Unlike classical protocols that synchronize:

- cryptographic states,

- clocks,
- sequence numbers,
- counters,

directional synchronization aligns internal geometric structures that are non-derivable.

It is:

- continuous,
- fractal,
- bidirectional,
- structural,
- living,
- oscillating (normal/ghost inversion),
- mutating (ID mutation).

6.2. The Three Tori and Their Roles in Synchronization

Synchronization relies on the constant interaction between:

6.2.1. The Public Torus

- Source of 3-bit blocks
- Universal directional base
- Contains all possible combinations of the 30 bits
- Determines orbital and fractal displacements
- Read across 10 successive tiers for every key

6.2.2. The Authentication Torus

- Used only once
- Validates the first key
- Initializes directional coherence
- Defines the first normal/ghost pairs
- Activates the first internal paths
- Creates the global ID

6.2.3. The Private Torus

- Reconstructed locally
- Never transmitted
- Evolves with every directional key

- Holds internal IDs, the global ID, and the torus's unique ID
- Undergoes normal/ghost inversion and ID mutation

Synchronization consists of maintaining perfect coherence between these three tori.

6.3. Triggering Synchronization

Synchronization begins immediately after the 5-transmission directional handshake (Chapter 5).

Once the first key is validated:

1. The Authentication Torus is read.
2. The Private Torus is initialized.
3. The first normal/ghost pairs are defined.
4. The first internal paths are activated.
5. The global ID is created.
6. The torus's unique ID is established.
7. The Glass Fortress is armed.
8. The Public Torus becomes the primary directional source.

From this moment on, each directional key maintains synchronization.

6.4. Fractal Synchronization: General Principle

Fractal synchronization relies on three fundamental rules:

6.4.1. Internal Coherence Rule

Each displacement must be coherent with:

- the normal/ghost bit,
- the internal path,
- the state of the Private Torus case,
- the associated internal ID,
- the global ID,
- the torus's unique ID.

6.4.2. Directional Continuity Rule

Progression must:

- remain within torus boundaries,
- follow internal paths,
- respect normal/ghost inversions,
- maintain coherence of internal IDs,

- maintain coherence of the global ID.

6.4.3. Fractal Expansion Rule

Exponential displacements (Path C):

- accelerate progression,
- create internal IDs,
- mutate existing IDs,
- increase directional complexity,
- reinforce fractal coherence.

6.5. Synchronization Between Public Torus and Private Torus

Synchronization between these two tori is the most important.

6.5.1. Reading 3-Bit Blocks

The 30 orbital bits of the directional key define:

- 60 orbital readings,
- 180 directional blocks (N,E),
- 180 exponential displacements.

Each displacement reads a 3-bit block:

- normal/ghost,
- internal path.

6.5.2. Updating the Private Torus

Each block read:

- updates a case in the Private Torus,
- inverts normal/ghost,
- activates an internal path,
- creates or validates an internal ID,
- mutates the starting case ID,
- reinforces the torus's unique ID.

6.5.3. Directional Coherence

Coherence is ensured if:

- displacements are compatible,
- internal paths are valid,
- internal IDs are coherent,

- the global ID remains stable,
- the torus's unique ID remains coherent.

6.6. Synchronization Between Authentication Torus and Private Torus

This synchronization occurs only once.

6.6.1. Initial Validation

The Authentication Torus:

- validates the first key,
- defines the first normal/ghost pairs,
- activates the first internal paths,
- initializes the global ID,
- initializes the torus's unique ID.

6.6.2. Transition to the Public Torus

Once the first key is validated:

- the Authentication Torus is no longer used,
- the Public Torus becomes the primary directional source.

6.7. Synchronization of Internal IDs

Internal IDs are created when:

- an internal path reaches a specific case,
- a coherent normal/ghost sequence is detected,
- a valid exponential progression is observed.

6.7.1. Updating IDs

Each key:

- creates new IDs,
- mutates existing IDs,
- updates the global ID,
- reinforces the torus's unique ID.

6.7.2. Role of IDs in Synchronization

IDs:

- guarantee fractal coherence,
- prevent derivations,

- prevent falsifications,
- prevent repetitions,
- stabilize directional progression.

6.8. Synchronization of the Glass Fortress

The Glass Fortress monitors:

- orbital displacements,
- normal/ghost pairs,
- internal paths,
- internal IDs,
- the global ID,
- the torus's unique ID,
- directional ports,
- the trusted IP.

It validates synchronization for every key.

6.9. Handling Synchronization Errors

6.9.1. Fractal Incoherence

If incoherence is detected:

- progression becomes impossible,
- an illegitimate path is found,
- an ID is incoherent,
- an illegitimate ID mutation occurs,
- an illegitimate normal/ghost inversion occurs,

the session is destroyed.

6.9.2. Incoherent Key

If a key:

- contains an invalid ID,
- contains an impossible starting point,
- contains an impossible ending point,
- contains an illegitimate type,

the session is destroyed.

6.9.3. No Response

If no response is received after 5 seconds:

- a new key is generated,
- with a different starting point,
- but reconstructing the same file.

6.10. Summary of Chapter 6

Directional synchronization:

- maintains coherence between the three tori,
- guarantees fractal progression,
- validates internal IDs,
- updates the global ID,
- stabilizes the torus's unique ID,
- ensures directional coherence,
- protects against derivations, falsifications, and repetitions,
- activates the Glass Fortress,
- destroys the session in case of anomaly.

It is the central mechanism that allows the protocol to function as an inviolable directional architecture.

Chapter 7 — Multi-Torus and Parallel Networks

7.1. Overview

The multi-torus system is one of the most powerful foundations of the directional architecture. It enables the creation of multiple independent directional networks, each possessing:

- its own Private Torus,
- its own Authentication Torus,
- its own directional keys,
- its own internal IDs,
- its own global ID,
- its own unique torus ID,
- its own Glass Fortress.

The Public Torus is pre-generated. It is included with the protocol software and serves as the universal directional base for all branches.

Each parallel network is completely isolated from the others, even if they coexist:

- on the same machine,
- on the same system,
- in the same network environment.

The multi-torus system enables:

- directional segmentation,
- structural sovereignty,
- total isolation of branches,
- creation of inviolable private networks,
- controlled directional duplication.

7.2. Definition of a Parallel Network

A parallel network is a complete instance of the directional architecture, comprising:

1. The Public Torus (pre-generated, universal)
2. An Authentication Torus
3. A Private Torus
4. A set of directional keys
5. A Glass Fortress
6. A global session ID
7. A unique torus ID

Each parallel network is an independent directional universe.

It shares:

- no keys,
- no IDs,
- no private torus,
- no fractal progression,
- no internal coherence.

7.3. Creation of a New Torus

Creating a new directional branch follows three steps:

7.3.1. Use of the Pre-Generated Public Torus

The Public Torus is not generated during branch creation. It is:

- pre-generated,
- provided with the software,
- universal,
- identical for all installations,
- non-modifiable,
- non-derivable.

It contains all possible combinations of the 30 bits, making it the universal directional base.

7.3.2. Generation of the Authentication Torus

An Authentication Torus of **1,000,000 cases × 3 bits** is generated. It validates the first directional key and initializes:

- the first normal/ghost pairs,
- the first internal paths,
- the global ID,
- the unique torus ID.

7.3.3. Generation of the Private Torus

A Private Torus of **1,000,000 cases × 3 bits** is generated. It will be reconstructed fractally from directional keys through:

- normal/ghost inversion,
- ID mutation,
- activation of internal paths,
- orbital progression.

Each Private Torus is independent from all existing private tori.

7.4. Directional Branches

Each parallel network is called a directional branch.

A branch possesses:

- its own Private Torus,
- its own fractal progression,
- its own internal IDs,
- its own Glass Fortress,
- its own global ID,

- its own unique torus ID.

7.4.1. Independent Branches

Branches do not communicate with each other. They cannot:

- share keys,
- share IDs,
- share private tori,
- share internal paths,
- share ID mutations.

7.4.2. Sovereign Branches

Each branch is sovereign:

- it does not depend on any other branch,
- it cannot be influenced by another branch,
- it cannot be derived by another branch.

7.5. Directional Duplication (Offline Primary Torus)

A key feature of the multi-torus system is offline directional duplication.

7.5.1. Primary Torus

A primary torus can be:

- generated,
- stored offline,
- transferred physically,
- used to create a new branch.

7.5.2. Offline Transfer

The primary torus can be transferred:

- on a USB drive,
- on a hard disk,
- on physical media,
- on any non-network medium.

This transfer contains no directional keys. It contains only:

- directional geometry,
- 3-bit blocks,
- internal paths.

7.5.3. Creating a Branch from the Primary Torus

Once the primary torus is transferred:

- a new branch is created,
- a new Private Torus is reconstructed,
- a new global ID is generated,
- a new unique torus ID is established,
- a new Glass Fortress is activated.

The branch is completely independent from the original branch.

7.6. Structural Isolation

Structural isolation is total.

7.6.1. Fractal Isolation

Fractal actions of one branch:

- cannot influence another branch,
- cannot derive another branch,
- cannot reconstruct another branch.

7.6.2. Directional Isolation

Internal paths of one branch:

- cannot be used in another branch,
- cannot be derived by another branch.

7.6.3. ID Isolation

Internal IDs:

- are not compatible between branches,
- cannot be transferred,
- cannot be derived.

7.7. Multi-Torus and Post-Quantum Security

The multi-torus system strengthens post-quantum security:

A quantum computer cannot:

- derive a private torus,
- derive a branch,
- derive a key,
- derive an ID,

- derive the unique torus ID.

Even with infinite power, an attacker cannot:

- explore a private torus,
- derive a private torus,
- reconstruct a private torus,
- analyze a private torus.

Private tori are geometrically inviolable.

7.8. Multi-Torus and Photonic Security

With the arrival of photonic computers:

- mathematical structures become obsolete,
- classical cryptography becomes vulnerable.

The multi-torus system is designed for this era:

Private tori are:

- non-transmissible,
- non-derivable,
- non-analyzable,
- non-explorable.

A photonic computer cannot:

- penetrate a private torus,
- analyze a private torus,
- derive a private torus.

7.9. Summary of Chapter 7

The multi-torus system enables:

- creation of parallel networks,
- total isolation of branches,
- offline directional duplication,
- structural sovereignty,
- post-quantum security,
- photonic security.

Each branch is an independent directional universe, inviolable by design. The pre-generated universal Public Torus serves as the common directional base, but never as a shared private torus.

Chapter 8 — Reconstruction of Directional Files

8.1. Overview

Directional file reconstruction is one of the major functions of the directional architecture. It enables reconstruction of a file bit by bit using:

- directional keys (64 bits),
- orbital displacements in the Public Torus,
- updates to the Private Torus,
- 3-bit blocks,
- internal IDs,
- the global ID,
- the torus's unique ID,
- directional coherence,
- the Glass Fortress.

Unlike classical systems that read a file sequentially, directional reconstruction is structural, geometric, non-linear, non-derivable, and inviolable by design.

Each directional key reconstructs exactly one bit of the file, following a fractal and orbital logic.

8.2. Types of Keys Involved in Reconstruction

File reconstruction primarily uses two types of keys:

Type	Bits	Description
00	File reconstruction	Reads the Public Torus to reconstruct one bit of the file
01	Followed key	Another key immediately follows to maintain coherence

Other types intervene in special cases:

Type	Bits	Description
11	Key reconstruction	Used to reconstruct other directional keys
10	Stop key	Ends reconstruction and stabilizes the torus

8.3. Breakdown of a Reconstruction Key (Type 00)

A reconstruction key contains:

- 8 bits: starting point (N^E)
- 8 bits: ending point (N^E)
- 2 bits: key type (00)
- 30 bits: orbital ring (navigation segment)

- 16 bits: targeted case ID

These elements define:

1. the directional window,
2. the zone of the Public Torus to traverse,
3. the zone of the Private Torus to update,
4. mutation of the starting case ID,
5. the file bit to reconstruct.

8.4. Reconstruction Process

Reconstruction follows a strict four-step process.

8.4.1. Step 1 — Determining the Directional Window

The 8 bits of the starting point and the 8 bits of the ending point define a directional window:

$$\mathbf{Window} = [N_1^{E_1} \rightarrow N_2^{E_2}]$$

This window determines:

- the zone of the Public Torus to traverse,
- the zone of the Private Torus to update,
- the zone of the file to reconstruct.

8.4.2. Step 2 — Orbital Navigation in the Public Torus

The 30 orbital bits of the key are transformed into:

- 60 orbital readings,
- 180 directional blocks (N,E),
- 180 exponential displacements.

Each displacement reads a 3-bit block in the Public Torus:

1. reading the normal/ghost bit,
2. reading the internal path (00, 01, 10, 11),
3. determining the next case,
4. influencing the Private Torus.

Orbital navigation is:

- non-linear,
- non-derivable,
- non-reversible,
- dependent on the internal state of the torus.

8.4.3. Step 3 — Updating the Private Torus

Each 3-bit block read in the Public Torus:

- updates a case in the Private Torus,
- inverts normal/ghost,
- activates an internal path,
- creates or validates an internal ID,
- mutates the starting case ID,
- reinforces the torus's unique ID.

The Private Torus gradually becomes coherent with the reconstructed file.

8.4.4. Step 4 — Reconstruction of the File Bit

Updates to the Private Torus allow reconstruction of one bit of the file.

The reconstructed bit depends on:

- the targeted case ID (16 bits),
- fractal coherence,
- directional progression,
- the internal state of the Private Torus,
- ID mutation,
- normal/ghost inversion.

Each key reconstructs exactly one bit.

8.5. Complete File Reconstruction

To reconstruct a file of **N bits**, the architecture uses **N type-00 keys**, optionally followed by type-01 keys.

8.5.1. Type-00 Keys

Each type-00 key reconstructs one bit.

8.5.2. Type-01 Keys

Type-01 keys indicate:

Another key immediately follows.

They allow:

- chaining keys,
- maintaining fractal coherence,
- ensuring directional continuity.

8.5.3. Type-10 Key (Stop)

A type-10 key:

- ends reconstruction,
- validates the reconstructed file,
- updates the global ID,
- stabilizes the Private Torus.

8.6. Protection Against Attacks

Directional reconstruction is protected by:

8.6.1. The Glass Fortress

It validates:

- keys,
- IDs,
- internal paths,
- normal/ghost pairs,
- fractal coherence,
- ID mutation.

8.6.2. The Tori

The tori are:

- non-derivable,
- non-transmissible,
- non-reversible,
- non-explorable.

8.6.3. Directional Keys

They are:

- unpredictable,
- unfalsifiable,
- non-injectable,
- non-repeatable.

8.6.4. Directional Destruction

In case of anomaly:

- the Private Torus is erased,

- IDs are invalidated,
- internal paths are destroyed,
- the global ID is erased,
- the session is terminated.

8.7. Summary of Chapter 8

Directional file reconstruction:

- uses type-00, type-01, and type-10 keys,
- reads the pre-generated Public Torus,
- updates the Private Torus,
- inverts normal/ghost,
- mutates internal IDs,
- reconstructs a file bit by bit,
- relies on directional geometry,
- is protected by the Glass Fortress,
- is inviolable by design.

It is one of the major applications of the directional architecture.

Chapter 9 — Reconstruction of Directional Keys (Type 11)

(Corrected version consistent with your directional mechanics)

9.1. Overview

Directional key reconstruction is an internal process that generates new keys from existing ones, following a fractal, orbital, and directional logic. This mechanism is activated by type-11 keys, which do not reconstruct a file but reconstruct other keys.

Key reconstruction is essential for:

- directional continuity,
- regeneration in case of key loss,
- creation of complex sequences,
- updating internal IDs,
- global fractal coherence,
- structural sovereignty of the torus.

It allows the protocol to operate without classical retransmission, without mathematical derivation, and without cryptography.

9.2. Role of Type-11 Keys

Type-11 keys have a unique function:

Reconstruct one or more directional keys from the internal fractal geometry.

They are used when:

- a key must be regenerated,
- a key must be extended,
- a key must be replaced,
- a key must be completed,
- a key must be reconstructed to maintain fractal coherence.

They never reconstruct a file. They reconstruct **keys**.

9.3. Structure of a Type-11 Key

A type-11 key has the same structure as all directional keys:

- 8 bits: starting point (N^E)
- 8 bits: ending point (N^E)
- 2 bits: key type (11)
- 30 bits: orbital ring (navigation segment)
- 16 bits: targeted case ID

✓ Integrated Correction

- The 30 bits always serve the escalation of the 10 orbital levels in the Public Torus.
- The starting and ending points serve **only** to determine the starting and ending points in the Private Torus.
- They have **no role** in the Public Torus.

9.4. Key Reconstruction Process

Directional key reconstruction follows a strict four-step process, identical to that of other key types.

9.4.1. Step 1 — Determining the Directional Window

The 8 bits of the starting point and the 8 bits of the ending point define a directional window in the Private Torus:

Window = $[N_1^E \rightarrow N_2^E]$

This window determines:

- the zone of the Private Torus to traverse,
- the zone of the Private Torus to update,
- the directional zone used to reconstruct the new key.

✓ Integrated Correction

The window **never** determines the zone of the Public Torus. The Public Torus is **always** traversed by the 30 orbital bits, independently of the starting point.

9.4.2. Step 2 — Orbital Navigation in the Public Torus

The 30 orbital bits of the key are transformed into:

- 10 orbital levels,
- 60 orbital readings,
- 180 directional blocks (N,E),
- 180 exponential displacements.

Each displacement reads a 3-bit block in the Public Torus:

1. reading the normal/ghost bit,
2. reading the internal path (00, 01, 10, 11),
3. determining the next case,
4. influencing the Private Torus.

Orbital navigation is:

- non-linear,
- non-derivable,
- non-reversible,
- dependent on the internal state of the torus.

9.4.3. Step 3 — Reading and Updating the Private Torus

Once the 10 orbital levels are executed, the key reads the cases of the Private Torus according to the directional window:

- updating normal/ghost pairs,
- activating internal paths,
- creating or validating internal IDs,
- mutating the starting case ID,
- reinforcing the torus's unique ID.

These updates do not reconstruct a file: they reconstruct an **internal directional state**.

9.4.4. Step 4 — Generating the New Key

Once the readings of the Public Torus and Private Torus are complete:

- the internal directional state is stabilized,
- internal IDs are updated,
- the global ID is adjusted,
- a new directional key is generated.

This new key:

- has a new starting point,
- a new ending point,
- a new type (00, 01, 10, or 11 depending on context),
- a new orbital navigation segment,
- a new targeted case ID.

It is structurally coherent with the previous key.

9.5. Special Case: Regeneration After Key Loss

If a key is lost (no response after 5 seconds), the protocol never retransmits the same key.

It uses a type-11 key to:

1. reconstruct a new key,
2. with a different starting point,
3. but reconstructing the same file,
4. or continuing the same fractal sequence.

Type-11 keys guarantee:

- fractal continuity,
- directional coherence,
- absence of collision,
- absence of derivability.

9.6. Special Case: Reconstruction of Complex Sequences

Type-11 keys are used to:

- reconstruct sequences of keys,
- extend sequences,
- create complex fractal sequences,
- stabilize directional sequences.

They allow creation of directional structures impossible to derive or falsify.

9.7. Protection Against Attacks

Key reconstruction is protected by:

9.7.1. The Glass Fortress

It validates:

- type-11 keys,
- internal IDs,
- internal paths,
- normal/ghost pairs,
- fractal coherence.

9.7.2. The Tori

The tori are:

- non-derivable,
- non-transmissible,
- non-reversible,
- non-explorable.

9.7.3. Directional Keys

They are:

- unpredictable,
- unfalsifiable,
- non-injectable,
- non-repeatable.

9.7.4. Directional Destruction

In case of anomaly:

- the Private Torus is erased,
- IDs are invalidated,
- internal paths are destroyed,
- the global ID is erased,
- the session is terminated.

9.8. Summary of Chapter 9

Directional key reconstruction:

- uses type-11 keys,
- interprets the 30 orbital bits,
- escalates the 10 levels of the Public Torus,
- then reads the Private Torus,
- inverts normal/ghost,
- mutates internal IDs,
- generates new keys,
- ensures fractal continuity,
- replaces lost keys,
- creates complex sequences,
- is protected by the Glass Fortress,
- is inviolable by design.

It is the internal mechanism that allows the protocol to operate without classical retransmission and without cryptography.

Chapter 10 — Stop Keys and End of Directional Sequence

10.1. Overview

Stop keys (type 10) are the terminal mechanism of a directional sequence. They do **not** reconstruct:

- files,
- keys,
- fractal sequences.

They have a single function:

To properly terminate a directional reconstruction and signal that the session must be closed.

Session closure then triggers an internal process called **directional destruction**, which is a structural mechanism independent from the key itself.

This chapter describes:

- the structure of a stop key,
- its fractal role,
- the conditions for ending a sequence,
- the directional closure process,
- the associated structural protections.

10.2. Structure of a Stop Key (Type 10)

A type-10 key has the same structure as all directional keys:

- 8 bits: starting point (N^E)
- 8 bits: ending point (N^E)
- 2 bits: key type (10)
- 30 bits: orbital ring (navigation segment)
- 16 bits: targeted case ID

✓ Integrated Correction

A type-10 key follows **exactly the same fractal steps** as all other keys:

1. Interpretation of the 30 orbital bits
2. Escalation through the 10 levels of the Public Torus
3. Reading blocks in the Private Torus
4. ID mutation
5. Normal/ghost inversion

The only difference lies in the **intent**:

- it reconstructs nothing,
- it generates no new key,
- it signals the end of the sequence.

10.3. Role of Stop Keys

Stop keys have three essential functions:

10.3.1. Terminating a Reconstruction

A type-10 key indicates:

The reconstruction is complete.

It validates:

- the end of a file,
- the end of a key sequence,
- the end of a directional branch.

10.3.2. Stabilizing the Global ID

Before closure, the global ID is:

- finalized,
- recorded,

- stabilized.

It becomes the terminal identifier of the session.

10.3.3. Signaling Directional Closure

A type-10 key indicates to the torus:

The session must be closed.

Directional closure is an internal mechanism triggered **after** the key.

10.4. Triggering Directional Closure

Directional closure is triggered when:

- a type-10 key is validated,
- the 5 handshake transmissions are respected,
- the Glass Fortress confirms fractal coherence,
- the global ID is stabilized.

If a type-10 key is incoherent:

- the session is closed immediately,
- but without stabilizing the global ID.

10.5. Directional Closure Process

Directional closure unfolds in four phases. *(Note: this process is internal. The type-10 key does not “perform” these actions — it triggers them.)*

10.5.1. Phase 1 — Voluntary Incoherence of Normal/Ghost Pairs

The normal/ghost pairs of the Private Torus are:

- inverted,
- scrambled,
- rendered incoherent.

10.5.2. Phase 2 — Invalidation of Internal Paths

Internal paths (00, 01, 10, 11) are:

- deactivated,
- made incompatible,
- made impossible to follow.

10.5.3. Phase 3 — Erasure of Internal IDs

Internal IDs:

- are erased,

- are rendered incoherent,
- lose their fractal relationships.

10.5.4. Phase 4 — Erasure of the Private Torus

The Private Torus is:

- erased,
- desynchronized,
- rendered structurally unreadable.

10.6. Protection Against Attacks

Directional closure protects against:

10.6.1. Post-Mortem Analysis

After closure:

- no ID is coherent,
- no internal path is valid,
- no normal/ghost pair is exploitable,
- no fractal progression is reconstructible.

10.6.2. Quantum Attacks

A quantum computer cannot:

- explore the torus,
- derive the torus,
- reconstruct the torus.

10.6.3. Photonic Attacks

A photonic computer cannot:

- analyze internal geometry,
- reconstruct internal paths,
- derive IDs.

10.7. Special Case: Forced Closure

Directional closure may be triggered **without** a type-10 key if:

- a fractal incoherence is detected,
- a key is falsified,
- a key is injected,
- a key is repeated,

- a key is derived,
- a key is incoherent.

In this case:

- the global ID is not stabilized,
- the session is closed immediately.

10.8. Summary of Chapter 10

Stop keys:

- terminate a reconstruction,
- stabilize the global ID,
- signal directional closure.

Directional closure:

- renders normal/ghost pairs incoherent,
- invalidates internal paths,
- erases internal IDs,
- erases the Private Torus,
- prevents any post-mortem analysis,
- protects against classical, quantum, and photonic attacks.

It is the terminal mechanism of the directional architecture, ensuring total structural inviolability.

Chapter 11 — Advanced Directional Transmission (Host ↔ Client ↔ Host ↔ Client ↔ Host)

(Modernized version consistent with Chapters 1 to 10)

11.1. Overview

Advanced directional transmission is the full evolution of the Host ↔ Client ↔ Host model introduced in Chapter 5. It formalizes:

- advanced transmission rules,
- fractal coherence conditions,
- directional correction mechanisms,
- structural protections,
- interactions with the tori,

- interactions with the Glass Fortress,
- special cases (loss, duplication, multi-torus),
- cascading transmissions,
- synchronized transmissions.

This chapter explains how a directional key actually circulates in a fractal environment—without cryptography, without derivation, and without classical retransmission.

11.2. Reminder of the Base Sequence (Modern Version)

Directional transmission always follows the full 5-step sequence:

1. **Host** → **Client**: sending the key
2. **Client** → **Host**: returning the same key
3. **Host** → **Client**: final return of the same key
4. **Client** → **Host**: confirmation of fractal execution
5. **Host** → **Client**: confirmation of torus synchronization

This sequence:

- validates the key,
- validates the client's IP,
- activates the Glass Fortress,
- confirms fractal execution,
- updates normal/ghost inversions,
- updates the starting case ID,
- synchronizes both Private Tori,
- continues fractal reconstruction.

11.3. Advanced Transmission: Structural Rules

11.3.1. Fractal Coherence Rule

A key is valid if:

- the 10 orbital levels are coherent,
- the 180 directional blocks are compatible,
- normal/ghost pairs are valid,
- internal paths are coherent,
- the targeted ID exists in the Private Torus,
- the global ID remains stable.

11.3.2. Directional Continuity Rule

The key must:

- continue fractal progression,
- respect ghost inversions,
- maintain coherence of internal IDs,
- correctly mutate the starting case ID.

11.3.3. Non-Repetition Rule

A key can **never** be repeated.

If an identical key is received twice:

- the session is destroyed,
- the Private Torus is erased,
- the IP is invalidated.

11.3.4. Non-Derivation Rule

A key cannot be derived:

- by an attacker,
- by an external torus,
- by another branch.

11.4. Advanced Transmission: Interactions with the Tori

11.4.1. Interaction with the Public Torus

The Public Torus:

- provides the 3-bit blocks,
- guides orbital displacements,
- influences reconstruction of the Private Torus.

Each transmission reads **180 blocks of 3 bits** (from the 30 orbital bits).

11.4.2. Interaction with the Private Torus

The Private Torus:

- validates displacements,
- updates normal/ghost pairs,
- creates internal IDs,
- mutates the starting case ID,
- updates the global ID.

Each transmission modifies **exactly one case** of the Private Torus.

11.4.3. Interaction with the Authentication Torus

The Authentication Torus:

- is used only once,
- validates the first key,
- initializes directional coherence.

11.5. Advanced Transmission: Interactions with the Glass Fortress

The Glass Fortress monitors:

- orbital displacements,
- normal/ghost pairs,
- internal paths,
- internal IDs,
- the global ID,
- directional ports,
- the trusted IP.

It validates or rejects each transmission.

11.5.1. Valid Transmission

A valid transmission:

- continues fractal progression,
- updates the Private Torus,
- stabilizes the global ID,
- maintains directional coherence.

11.5.2. Invalid Transmission

An invalid transmission:

- destroys the session,
- erases the Private Torus,
- invalidates the IP,
- deactivates internal paths.

11.6. Advanced Transmission: Special Cases

11.6.1. Key Loss

If no response after 5 seconds:

- the key is considered lost,
- a new key is generated,
- with a different starting point,
- but reconstructing the same file.

No retransmission of the same packet is ever performed.

11.6.2. Cascading Transmission

Some sequences require:

- a type-01 key (another key follows),
- followed by a type-00 or type-11 key.

Cascading transmission:

- maintains fractal coherence,
- enables complex reconstructions,
- stabilizes internal IDs.

11.6.3. Multi-Torus Transmission

In a multi-torus environment:

- each branch has its own transmission,
- transmissions never cross,
- keys are not compatible between branches.

A key from one branch cannot be used in another.

11.6.4. Synchronized Transmission

Some sequences require synchronization:

- between multiple keys,
- between multiple IDs,
- between multiple internal paths.

Synchronized transmission:

- stabilizes the global ID,
- creates complex fractal structures,
- enables advanced directional reconstructions.

11.7. Advanced Transmission: Structural Protections

11.7.1. Classical Attacks

Impossible to:

- falsify a key,
- inject a key,
- repeat a key,
- derive a key.

11.7.2. Quantum Attacks

Impossible to:

- explore the torus,
- derive the torus,
- reconstruct the torus.

11.7.3. Photonic Attacks

Impossible to:

- analyze internal geometry,
- reconstruct internal paths,
- derive IDs.

11.8. Summary of Chapter 11

Advanced directional transmission:

- extends the Host $\leftrightarrow$ Client $\leftrightarrow$ Host $\leftrightarrow$ Client $\leftrightarrow$ Host model,
- adds fractal rules,
- maintains directional coherence,
- protects against derivation, falsification, and repetition,
- interacts with all three tori,
- interacts with the Glass Fortress,
- handles loss, cascades, and synchronization,
- operates in multi-torus environments,
- is inviolable by design.

It is the fundamental communication mechanism of the directional architecture.

Chapter 12 — Directional Ports and Internal Communication Geometry

12.1. Overview

Directional ports are one of the most unique and revolutionary aspects of the directional architecture. Unlike classical ports (TCP/UDP), which are integers between 0 and 65535, directional ports are internal geometric structures defined by:

- internal paths (00, 01, 10, 11),
- normal/ghost pairs,
- internal IDs,
- the global ID,
- the fractal position within the Private Torus,
- the directional window ($\mathbb{N}^E \rightarrow \mathbb{N}^E$),
- directional coherence validated by the Glass Fortress.

A directional port is **not**:

- a number,
- a network identifier,
- a TCP channel,
- a socket,
- scannable.

A directional port is a living internal geometry, impossible to derive, explore, or falsify.

12.2. Definition of a Directional Port

A directional port is a fractal structure composed of:

1. An internal path (00, 01, 10, 11)
2. A normal/ghost pair
3. An internal ID
4. A global ID
5. A fractal position within the Private Torus
6. A directional window ($\mathbb{N}^E \rightarrow \mathbb{N}^E$)
7. Fractal coherence validated by the Glass Fortress

A directional port is therefore a **geometric object**, not a numerical one.

12.3. Why Directional Ports Are Inviolable

12.3.1. They Do Not Exist in the Classical Network Layer

An attacker cannot:

- scan ports,
- probe ports,
- send packets to a port,
- open a port,
- close a port.

Directional ports exist **only** within the internal geometry of the torus.

12.3.2. They Are Defined by Fractal Mechanics

A directional port depends on:

- the 10 orbital levels,
- the 180 directional blocks,
- the 3-bit blocks,
- internal paths,
- normal/ghost pairs,
- internal IDs.

Impossible to derive.

12.3.3. They Are Validated by the Glass Fortress

The Glass Fortress:

- monitors directional ports,
- validates their coherence,
- destroys the session if a port is violated.

12.3.4. They Are Non-Transmissible

A directional port:

- cannot be sent,
- cannot be copied,
- cannot be exported,
- cannot be derived.

It exists only locally, within the torus.

12.4. Internal Structure of a Directional Port

A directional port is composed of seven fractal layers.

12.4.1. Layer 1 — Internal Path

The internal path (00, 01, 10, 11) determines:

- the port's direction,
- the port's geometry,
- the port's fractal progression.

12.4.2. Layer 2 — Normal/Ghost Pair

The normal/ghost pair determines:

- the nature of the port,
- the validity of the port,
- the coherence of the port.

A ghost port is inverted. A normal port is direct.

12.4.3. Layer 3 — Internal ID

Each port has an internal ID created by:

- orbital displacements,
- 3-bit blocks,
- internal paths.

This ID is unique, non-derivable, non-transmissible.

12.4.4. Layer 4 — Global ID

The global ID:

- links the port to the session,
- stabilizes internal geometry,
- prevents collisions.

12.4.5. Layer 5 — Position in the Private Torus

Each port is associated with a case in the Private Torus:

- 1,000,000 cases,
- internal directional structure.

The port's position is fractal, non-linear.

12.4.6. Layer 6 — Directional Window

The directional window ($N^E \rightarrow N^E$) determines:

- the zone of the Private Torus used,
- the port's fractal zone,
- mutation of the starting case ID.

(Note: the window never determines the zone of the Public Torus.)

12.4.7. Layer 7 — Fractal Coherence

Fractal coherence is validated by:

- the 10 orbital levels,
- normal/ghost pairs,
- internal paths,
- internal IDs,
- the global ID.

An incoherent port destroys the session.

12.5. Directional Ports and Transmission

Directional ports intervene in transmission:

12.5.1. Host → Client

The host opens an internal directional port. The client must respond on the same directional port.

12.5.2. Client → Host

The client returns the key through this port. The host validates fractal coherence.

12.5.3. Host → Client

The host returns the final key through the same port.

If a directional port is violated:

- the session is destroyed,
- the Private Torus is erased,
- the IP is invalidated.

12.6. Directional Ports and Multi-Torus Environments

In a multi-torus environment:

- each branch has its own directional ports,
- ports are not compatible between branches,
- ports cannot be derived from one branch to another.

A directional port is strictly local to its branch.

12.7. Directional Ports and Post-Quantum Security

Directional ports are inviolable by a quantum computer:

- impossible to derive internal geometry,
- impossible to explore the torus,
- impossible to reconstruct internal paths,
- impossible to derive IDs.

12.8. Directional Ports and Photonic Security

Photonic computers can:

- analyze mathematical structures,
- derive cryptographic keys,
- break classical protocols.

But they cannot:

- analyze directional geometry,
- derive a torus,
- reconstruct a directional port.

Directional ports are structurally inviolable.

12.9. Summary of Chapter 12

Directional ports:

- are not TCP ports,
- are internal fractal structures,
- are defined by internal paths,
- are defined by normal/ghost pairs,
- are defined by internal IDs,
- are validated by the Glass Fortress,
- are inviolable by design,
- cannot be scanned,
- cannot be derived,
- cannot be falsified.

They form the internal communication layer of the directional architecture, completely independent from classical network protocols.

Chapter 13 — Internal Geometry of the Private Torus (Living Structure)

13.1. Overview

The Private Torus is the living organ of TorusNet. Unlike:

- the Public Torus (static, universal, pre-generated),
- the Authentication Torus (static, used only once),

the Private Torus is:

- living,
- evolving,
- directional,
- fractal-geometric,
- non-derivable,
- non-transmissible,
- non-reversible,
- non-analyzable,
- non-explorable,
- not explorable even by a quantum or photonic computer.

It constitutes the sovereign internal structure of every TorusNet session.

13.2. Basic Structure of the Private Torus

The Private Torus is a grid of:

1,000,000 cases (1000 × 1000)

Each case contains:

- a normal or ghost state,
- 4 predefined internal paths,
- a case ID (16 bits),
- a relationship with the global ID.

✓ Fundamental Rule

The 3 bits produced by the escalation of the 10 orbital levels are interpreted as follows:

- **Bit 0** → indicates whether the ghost bit must be read
- **Bits 1–2** → select one of the 4 internal paths of the case

Thus, each case is a directional node with 4 exits, controlled by fractal geometry.

13.3. The Private Torus as a Living Structure

13.3.1. It Evolves with Every Directional Key

Each key:

- selects an internal path via the 3 orbital bits,
- modifies a case,
- modifies the normal/ghost state,
- activates one of the 4 internal paths,
- creates or validates an internal ID,
- mutates the starting case ID,
- updates the global ID.

13.3.2. It Possesses Directional Memory

This memory is geometric:

- in normal/ghost states,
- in the 4 internal paths,
- in internal IDs,
- in the global ID.

13.3.3. It Possesses Internal Coherence

Internal coherence is:

- fractal,
- directional,
- non-linear,
- non-mathematical.

13.3.4. It Possesses Internal Dynamics

The Private Torus:

- moves,
- reorganizes,
- rebalances,
- stabilizes,
- destroys itself.

13.4. The Three Internal Layers of the Private Torus

13.4.1. Layer 1 — Normal/Ghost Geometry

Defines:

- the nature of displacements,
- the validity of paths,
- fractal coherence.

A ghost state is a directional inversion.

13.4.2. Layer 2 — Internal Paths (4 per Case)

Each case contains 4 internal paths:

- a local path,
- a diagonal path,
- an exponential path,
- a complex directional path.

The 3 orbital bits select the path:

- **Bit 0** → **ghost reading activated or not**
- **Bits 1–2** → **internal direction (00, 01, 10, 11)**

The **10 path (exponential)** is the most powerful → it creates internal IDs.

13.4.3. Layer 3 — Internal IDs and Global ID

Internal IDs are created when:

- an internal path reaches a specific case,
- a coherent normal/ghost sequence is detected,
- a valid exponential progression is observed.

Directional Partition of the 16 Bits

The 16-bit case ID is divided into two groups:

1. **32,768 values** → **valid starting cases**
2. **32,768 values** → **directional ID permutations**

Definition of the New Case ID

When a key is executed:

- the first elements reconstructed in the Private Torus define the new case ID.

This new ID:

- replaces the old one,

- is chosen among the 32,768 permutation values,
- depends on the fractal geometry produced by the key,
- is impossible to predict or derive.

Global ID

The global ID:

- summarizes the state of the torus,
- stabilizes the session,
- guarantees directional coherence.

13.5. Internal Geometry: Fractal Dynamics

13.5.1. Coherence Rule

A displacement is valid if:

- the normal/ghost state is coherent,
- the selected internal path is compatible,
- the internal ID is valid,
- the global ID remains stable.

13.5.2. Continuity Rule

Progression must:

- remain within torus boundaries,
- follow internal paths,
- respect ghost inversions.

13.5.3. Expansion Rule

Exponential displacements:

- accelerate progression,
- create internal IDs,
- increase fractal complexity.

13.6. Interaction with Directional Keys

Each directional key:

- interprets the 30 orbital bits,
- escalates the 10 levels of the Public Torus,
- produces a directional sequence divided into 3-bit blocks,
- **Bit 0** → indicates whether the ghost bit must be read,

- **Bits 1–2 → select one of the 4 internal paths,**
- updates a case in the Private Torus,
- uses the 16 bits to determine the starting case,
- uses the first reconstructed elements to define the new case ID,
- replaces the old ID,
- modifies internal geometry,
- creates or validates internal IDs,
- updates the global ID.

The Private Torus is entirely reconstructed by the keys.

13.7. Interaction with the Glass Fortress

The Glass Fortress monitors:

- normal/ghost states,
- internal paths,
- internal IDs,
- the global ID,
- directional ports.

If an incoherence is detected:

- the session is destroyed,
- the Private Torus is erased.

13.8. Interaction with Directional Ports

Directional ports are internal structures of the Private Torus.

They depend on:

- the 4 internal paths,
- normal/ghost states,
- internal IDs,
- the global ID.

A directional port is an internal geometry, not a number.

13.9. Interaction with Multi-Torus Environments

Each branch has its own Private Torus.

Private tori:

- are not compatible between branches,

- cannot be derived,
- cannot be transferred,
- cannot be analyzed.

Each Private Torus is an independent directional universe.

13.10. Directional Destruction of the Private Torus

Directional destruction:

- renders normal/ghost pairs incoherent,
- invalidates internal paths,
- erases internal IDs,
- erases the global ID,
- erases the Private Torus.

It prevents any post-mortem analysis.

13.11. Summary of Chapter 13

The Private Torus:

- is a living structure,
- evolves with every key,
- has 4 internal paths per case,
- uses the 3 orbital bits to select the path,
- possesses normal/ghost states,
- possesses internal IDs,
- possesses a global ID,
- mutates the starting case ID with every key,
- defines the new ID via the first reconstructed elements,
- interacts with keys,
- interacts with the Glass Fortress,
- interacts with directional ports,
- is inviolable by design,
- is destroyed in case of anomaly.

It is the living core of TorusNet.

Chapter 14 — Internal Geometry of the Public Torus (Static Directional Structure)

(Version: the Public Torus contains all possible combinations of 30 bits)

14.1. Overview

The Public Torus is the static and exhaustive directional structure of TorusNet. Unlike:

- the Private Torus (living, evolving),
- the Authentication Torus (used only once),

the Public Torus is:

- immutable,
- universal,
- complete,
- non-modifiable,
- non-reversible,
- non-derivable,
- non-transmissible,
- non-analyzable,
- non-explored,
- not explorable even by a quantum or photonic computer.

It constitutes the total directional source of all keys, all fractal progressions, all reconstructions, and all structural validations.

14.2. Basic Structure of the Public Torus

The Public Torus contains:

all possible combinations of 30 bits $\rightarrow 2^{30} = 1,073,741,824$ unique directional cases

Each case contains exactly 30 bits, divided into:

- **10 blocks of 3 bits,**
- **no unused bits.**

Interpretation of the 3 Bits

Each 3-bit block follows the directional rule:

- **Bit 0:** normal (0) or ghost (1)
- **Bits 1–2:** internal direction (00, 01, 10, 11)

These blocks define:

- fractal selectors,
- normal/ghost inversions,
- internal directions,
- progression signals,
- ID mutations in the Private Torus,
- global directional coherence.

14.3. Static Nature of the Public Torus

The Public Torus is completely static, meaning:

14.3.1. It Never Changes

No directional key can:

- modify the Public Torus,
- alter the Public Torus,
- rewrite the Public Torus.

14.3.2. It Does Not Depend on Any Session

The Public Torus:

- is identical for all sessions,
- is identical for all clients,
- is identical for all hosts.

14.3.3. It Does Not Depend on Any Branch

In a multi-torus environment:

- each branch uses the same Public Torus,
- the Public Torus remains static.

14.3.4. It Does Not Depend on Any Key

Directional keys:

- read the Public Torus,
- but never modify it.

14.4. Internal Geometry of the Public Torus

The internal geometry of the Public Torus is defined by:

1. the 10 blocks of 3 bits,
2. normal/ghost inversions,
3. internal directions (00, 01, 10, 11),

4. fractal sequences,
5. directional windows ($N^E \rightarrow N^E$),
6. orbital signals produced by keys.

This geometry is:

- non-linear,
- non-mathematical,
- non-derivable,
- non-reversible,
- non-explored,
- not explorable even by a quantum or photonic computer.

14.5. Role of the Public Torus in Fractal Mechanics

The Public Torus is the total directional source of fractal mechanics.

Each directional key:

- interprets the 30 navigation bits,
- escalates the 10 orbital levels,
- produces 10 blocks of 3 bits,
- reads normal/ghost via bit 0,
- reads internal direction via bits 1–2,
- transmits these signals to the Private Torus,
- triggers internal updates,
- creates or validates internal IDs,
- updates the global ID.

The Public Torus is therefore the static directional base of TorusNet.

14.6. Interaction Between Public Torus and Private Torus

The relationship between the two tori is **asymmetric**:

14.6.1. The Public Torus Influences the Private Torus

The Public Torus:

- guides fractal displacements,
- influences updates to the Private Torus,
- influences creation of internal IDs,
- influences mutation of case IDs,

- influences updates to the global ID.

14.6.2. The Private Torus Never Influences the Public Torus

The Private Torus:

- cannot modify the Public Torus,
- cannot derive the Public Torus,
- cannot reconstruct the Public Torus.

14.7. Interaction Between Public Torus and Directional Keys

Directional keys use the Public Torus to:

- reconstruct files,
- reconstruct keys,
- reconstruct fractal sequences,
- stabilize the global ID,
- validate directional coherence.

The 30 navigation bits of a key:

- are divided into 10 blocks of 3 bits,
- each block selects an internal path in the Private Torus,
- each block influences fractal progression.

14.8. Interaction Between Public Torus and the Glass Fortress

The Glass Fortress uses the Public Torus to:

- validate orbital signals,
- validate internal directions,
- validate normal/ghost inversions,
- validate internal IDs,
- validate the global ID.

If an incoherence is detected:

- the session is destroyed,
- the Private Torus is erased.

14.9. Interaction Between Public Torus and Multi-Torus Environments

In a multi-torus environment:

- each branch uses the same Public Torus,

- the Public Torus is static,
- the Public Torus is independent,
- the Public Torus is inviolable.

14.10. Summary of Chapter 14

The Public Torus:

- is static,
- is universal,
- is immutable,
- contains all possible combinations of 30 bits,
- i.e., **1,073,741,824 directional cases**,
- contains 10 blocks of 3 bits per case,
- defines directional geometry,
- guides fractal displacements,
- influences the Private Torus,
- is validated by the Glass Fortress,
- is non-derivable,
- non-transmissible,
- non-reversible,
- non-analyzable,
- non-explored,
- not explorable even by a quantum or photonic computer.

It constitutes the total static directional base of TorusNet.

Chapter 15 — Internal Geometry of the Authentication Torus (Initial Directional Structure)

(Version: same case structure as the Private Torus)

15.1. Overview

The Authentication Torus is the initial directional structure of TorusNet. Unlike:

- the Public Torus (static, universal, exhaustive),
- the Private Torus (living, evolving),

the Authentication Torus is:

- static,
- local,
- unique to each branch,
- used only once,
- non-modifiable,
- non-derivable,
- non-transmissible,
- non-reversible,
- non-analyzable,
- non-explored,
- not explorable even by a quantum or photonic computer.

It forms the directional entry point of TorusNet. It validates the first directional key and initializes fractal coherence.

15.2. Basic Structure of the Authentication Torus

The Authentication Torus has **exactly the same structure as the Private Torus**:

1,000,000 cases (1000 × 1000) Each case contains:

- a normal/ghost state,
- 4 predefined internal paths,
- a case ID (16 bits),
- a relationship with the global ID,
- **10 blocks of 3 bits** serving as directional selectors.

Interpretation of the 3 Bits

Each 3-bit block follows the directional rule:

- **Bit 0:** indicates whether the ghost bit must be read
- **Bits 1–2:** select one of the 4 internal paths of the case

These blocks define:

- the initial directional geometry,
- the first fractal displacements,
- the first normal/ghost inversions,
- the first internal paths,
- the first internal IDs,

- the initial global ID.

15.3. Role of the Authentication Torus

The Authentication Torus has three essential roles:

15.3.1. Validate the First Directional Key

The first directional key is validated through:

- the 3-bit blocks of the Authentication Torus,
- the selected internal paths,
- normal/ghost inversions,
- the first internal IDs,
- initial fractal coherence.

15.3.2. Initialize the Private Torus

The Authentication Torus:

- initializes the normal/ghost states of the Private Torus,
- initializes the 4 internal paths of each case,
- initializes internal IDs,
- initializes the global ID.

15.3.3. Activate the Glass Fortress

The Glass Fortress is activated when:

- the first key is validated,
- initial fractal coherence is confirmed,
- the client's IP becomes a trusted IP.

15.4. Internal Geometry of the Authentication Torus

The internal geometry of the Authentication Torus is defined by:

1. the 10 blocks of 3 bits,
2. internal directions (00, 01, 10, 11),
3. normal/ghost inversions,
4. initial fractal sequences,
5. directional windows ($N^E \rightarrow N^E$),
6. orbital signals of the first key.

This geometry is:

- non-linear,

- non-mathematical,
- non-derivable,
- non-reversible,
- non-explored,
- not explorable even by a quantum or photonic computer.

15.5. Interaction Between Authentication Torus and First Key

The first directional key:

- interprets the 30 orbital bits,
- escalates the 10 directional levels,
- produces 10 blocks of 3 bits,
- reads normal/ghost via bit 0,
- reads internal direction via bits 1–2,
- initializes the Private Torus,
- creates the first internal IDs,
- creates the initial global ID.

The Authentication Torus is therefore the initial directional base of TorusNet.

15.6. Interaction Between Authentication Torus and Private Torus

The relationship between the two tori is **asymmetric**:

15.6.1. The Authentication Torus Influences the Private Torus

The Authentication Torus:

- initializes normal/ghost states,
- initializes the 4 internal paths,
- initializes internal IDs,
- initializes the global ID.

15.6.2. The Private Torus Never Influences the Authentication Torus

The Private Torus:

- cannot modify the Authentication Torus,
- cannot derive the Authentication Torus,
- cannot reconstruct the Authentication Torus.

15.7. Interaction Between Authentication Torus and the Glass Fortress

The Glass Fortress uses the Authentication Torus to:

- validate the first key,
- validate the first fractal displacements,
- validate normal/ghost inversions,
- validate the first internal paths,
- validate the first internal IDs,
- validate the initial global ID.

If an incoherence is detected:

- the session is destroyed,
- the Private Torus is erased.

15.8. Interaction Between Authentication Torus and Multi-Torus Environments

In a multi-torus environment:

- each branch has its own Authentication Torus,
- each Authentication Torus is static,
- each Authentication Torus is independent,
- each Authentication Torus is inviolable.

15.9. Summary of Chapter 15

The Authentication Torus:

- is static,
- is local,
- is unique to each branch,
- has exactly the same case structure as the Private Torus,
- contains **1,000,000 cases**,
- each case contains:
 - a normal/ghost state,
 - 4 internal paths,
 - a case ID,
 - a relationship with the global ID,

- 10 blocks of 3 bits,
- defines the initial directional geometry,
- validates the first key,
- initializes the Private Torus,
- creates the first internal IDs,
- creates the global ID,
- activates the Glass Fortress,
- is non-derivable,
- non-transmissible,
- non-reversible,
- non-analyzable,
- non-explored,
- not explorable even by a quantum or photonic computer.

It constitutes the initial directional structure of TorusNet.

Chapter 16 — Global Directional Coherence (Global ID and Fractal Stability)

16.1. Overview

Global directional coherence is the mechanism that ensures the entire TorusNet session remains:

- stable,
- coherent,
- inviolable,
- non-derivable,
- non-reversible,
- non-falsifiable,
- non-analyzable,
- not explorable even by a quantum or photonic computer.

This mechanism relies on a central element:

the Global ID, a living directional structure that summarizes the complete fractal state of the Private Torus.

The Global ID is **not**:

- a number,

- a key,
- a hash,
- a signature,
- a mathematical structure.

It is an **emergent directional structure**, derived from:

- the internal geometry of the Private Torus,
- normal/ghost pairs,
- the 4 internal paths,
- internal IDs,
- directional windows,
- fractal displacements,
- validated directional keys,
- the Glass Fortress.

16.2. Definition of the Global ID

The Global ID is a living fractal structure, but its directional encoding is precisely defined:

The Global ID is a 16-bit directional ID, constructed from the second segment of the fractal traversal in the Private Torus.

- The **first 16-bit segment** reconstructed defines the **starting case ID**.
- The **second 16-bit segment** defines the **Global ID**.

The two segments are successive, independent, and produced by the same traversal.

Origin of the Two Segments

- **Bits 0–15** → **starting case ID** (first fractal window)
- **Bits 16–31** → **Global ID** (second fractal window)

Value Space

- Case IDs use their own 16-bit ranges.
- The Global ID uses a free value among the 16 bits not used by case IDs.

Thus, the Global ID is:

- a 16-bit directional ID,
- living,
- updated with every key,
- impossible to derive or replay.

16.3. Role of the Global ID

16.3.1. Fractal Coherence of the Session

It ensures that:

- keys are coherent,
- fractal displacements are coherent,
- internal IDs are coherent,
- internal paths are coherent,
- normal/ghost pairs are coherent.

16.3.2. Directional Stability

It stabilizes:

- fractal progression,
- updates to the Private Torus,
- directional ports,
- key sequences.

16.3.3. Structural Protection

It prevents:

- derivations,
- falsifications,
- repetitions,
- injections,
- illegitimate reconstructions.

16.3.4. Directional Destruction

In case of anomaly:

- the Global ID is invalidated,
- the session is destroyed,
- the Private Torus is erased.

16.4. Creation of the Global ID

The Global ID is created during validation of the first directional key, using the Authentication Torus.

Creation relies on:

1. the first 16-bit segment (case ID),

2. the second 16-bit segment (Global ID),
3. initial normal/ghost pairs,
4. initial internal paths,
5. the first internal IDs,
6. the first directional window $N^E \rightarrow N^E$,
7. validation by the Glass Fortress.

The Global ID is a **fractal emergence**, not a calculation.

16.5. Updating the Global ID

Each directional key updates the Global ID in two successive steps:

16.5.1. Reconstruction of the First ID (Case ID)

The fractal traversal reconstructs a 16-bit ID:

ID_case = bits 0–15

16.5.2. Reconstruction of the Second ID (Global ID)

The traversal continues immediately and reconstructs a second 16-bit ID:

ID_global = bits 16–31

16.5.3. Directional Validation

The Glass Fortress:

- validates both IDs,
- validates fractal coherence,
- validates directional coherence,
- stabilizes the Global ID.

16.6. Fractal Stability

Fractal stability is ensured by:

- coherence of normal/ghost pairs,
- coherence of internal paths,
- coherence of internal IDs,
- coherence of directional windows,
- coherence of fractal displacements.

16.7. Interaction Between Global ID and the Glass Fortress

The Glass Fortress:

- monitors the Global ID,
- validates the Global ID,
- stabilizes the Global ID,
- destroys the session if the Global ID is incoherent.

The Global ID is the keystone of directional security.

16.8. Interaction Between Global ID and Multi-Torus Environments

In a multi-torus environment:

- each branch has its own Global ID,
- Global IDs are not compatible between branches,
- Global IDs cannot be derived,
- Global IDs cannot be transferred.

Each Global ID is an independent directional universe.

16.9. Directional Destruction and the Global ID

During directional destruction:

- the Global ID is invalidated,
- internal IDs are erased,
- internal paths are destroyed,
- normal/ghost pairs become incoherent,
- the Private Torus is erased.

The Global ID is the **first element destroyed**.

16.10. Summary of Chapter 16

Global directional coherence relies on:

- the Global ID,
- internal IDs,
- internal paths,
- normal/ghost pairs,
- directional windows,
- fractal displacements,
- validation by the Glass Fortress.

The Global ID:

- is a living fractal structure,
- is a 16-bit directional ID,
- is produced by the second segment of the fractal traversal,
- uses a free value not used by case IDs,
- summarizes the state of the Private Torus,
- stabilizes the session,
- protects against attacks,
- is inviolable by design,
- is destroyed in case of anomaly.

It is the central directional structure of TorusNet.

Chapter 17 — Logic of Torus Reading (Private and Public)

17.1. Overview

Reading the toruses is a fundamental element of the TorusNet protocol. It determines:

- fractal progression,
- directional coherence,
- file reconstruction,
- key reconstruction,
- mutation of internal IDs,
- Host ↔ Client ↔ Host synchronization.

This chapter describes the complete reading logic of:

- the Private Torus,
- the Public Torus,
- exponential displacements,
- directional displacements,
- the torus's circular behavior,
- full-column reading.

17.2. Reading the Private Torus

The Private Torus is a closed directional space. Its reading depends on the type of displacement defined by the key:

- exponential displacements,
- directional displacements (up, down, left, right, diagonals).

Both modes are complementary.

17.2.1. Exponential Reading: Circular Line-by-Line Mode

When the key's starting point is read and the displacement cases produce an exponential value, then:

The Private Torus is read in a circular, line-by-line manner.

Circular Reading Rules

- The torus is traversed line by line, from left to right.
- When reading reaches the end of a line, it wraps to the beginning of the next line.
- When reading reaches the last line, it wraps back to the first line.
- The torus is a cyclic space: no edges, no end, no boundaries.

Directional Effects

This mode allows:

- reaching any case in the torus,
- creating long fractal sequences,
- ensuring perfect directional coherence,
- avoiding directional collisions.

17.2.2. Directional Reading: Up, Down, Left, Right, Diagonals

When displacements are:

- up,
- down,
- left,
- right,
- diagonal,

the Private Torus behaves like a real topological torus.

Rule: Directional Wrap-Around

- Exceed upward → reappear at the bottom.
- Exceed downward → reappear at the top.
- Exceed left → reappear on the right.
- Exceed right → reappear on the left.
- Diagonals combine these rules.

Origin of the Name TorusNet

This wrap-around principle is precisely what defines a topological torus. It is the reason the protocol is named **TorusNet**: a network whose internal geometry behaves like a torus, where every overflow naturally leads to the opposite side.

Directional Effects

This mode enables:

- coherent local displacements,
- stable internal paths,
- precise normal/ghost inversions,
- controlled fractal progression.

17.3. Reading the Public Torus

The Public Torus is read using a different logic. The key's 30 bits represent **1000 fractal displacements**, each producing a column value.

Rule: Full-Column Reading

For each fractal displacement:

1. The 30 bits are interpreted by the internal algorithm.
2. The algorithm determines a column of the Public Torus.
3. The program reads the entire column from top to bottom.
4. The 3 bits of each case are used to:
 - determine normal/ghost inversions,
 - determine internal paths,
 - determine ID mutations,
 - determine fractal displacements in the Private Torus.

Directional Effects

This mode enables:

- massive directional extraction,
- global fractal coherence,
- deep directional progression,
- file or key reconstruction.

17.4. Interaction with Directional Updates

MAJ 1 — Inversion and ID Mutation

Reading the torus triggers:

- normal/ghost inversion,
- mutation of the starting case's ID.

MAJ 2 — Removal of Automatic Renewal

Torus reading is no longer interrupted by a timed cycle. It is continuous, living, directional.

MAJ 3 — Unique Torus ID

If a key contains the unique torus ID:

- the Client detects a directional renewal,
- torus reading switches into controlled reset mode.

17.5. Summary of Chapter 17

Torus reading relies on:

- a circular mode for exponential displacements,
- a wrap-around mode for directional displacements,
- full-column reading for the Public Torus,
- total directional coherence,
- living fractal progression,
- Host ↔ Client ↔ Host synchronization.

This chapter completes the fractal mechanics and stabilizes the internal logic of the TorusNet protocol.

Chapter 18 — DNS and HTTPS Authentication (Temporary Trust Layer)

18.1. Overview

Before TorusNet's directional geometry takes full control of the session, a temporary trust layer is required to:

- identify the requested host,
- avoid malicious redirections,
- prevent classic MITM attacks,
- secure the transmission of the first directional key,
- validate that the client is indeed contacting the correct server.

This layer relies on two classical Internet technologies:

- **DNS** (Domain Name System),

- **HTTPS** (TLS certificate).

These technologies are **not** part of the directional mechanics. They serve only as an **entry airlock**, before TorusNet activates:

- the Authentication Torus,
- the Private Torus,
- the Public Torus,
- the Glass Fortress,
- internal IDs,
- the global ID.

Once the first key is validated, DNS and HTTPS disappear completely.

18.2. Role of DNS in TorusNet

DNS is used for one purpose only:

Identify the requested host before entering directional geometry.

18.2.1. Domain Verification

When a client contacts a server:

1. DNS resolves the domain name.
2. TorusNet checks whether this domain belongs to the trusted host list.
3. If the domain is unknown or suspicious:
 - the connection switches to **user confirmation mode**,
 - no directional key is transmitted.

18.2.2. Protection Against Redirections

DNS prevents:

- malicious redirections,
- fake domains,
- cloned hosts,
- traffic hijacking.

18.2.3. Strictly Temporary Role

Once the first key is validated:

DNS is no longer used. Directional geometry takes over.

18.3. Role of HTTPS in TorusNet

HTTPS is used to:

secure the transmission of the first directional key.

18.3.1. Certificate Verification

The client verifies:

- the TLS certificate,
- the domain's validity,
- the absence of classic MITM interception.

If the site does not support HTTPS:

- the connection switches to **user confirmation mode**,
- no directional key is sent.

18.3.2. Protection Against Classic Attacks

HTTPS protects against:

- interceptions,
- injections,
- traffic manipulation,
- downgrade attacks.

18.3.3. Strictly Temporary Role

Once the first key is transmitted:

HTTPS disappears. The session becomes 100% directional.

18.4. DNS ↔ HTTPS ↔ Authentication Torus Interaction

The complete sequence is:

1. **DNS** identifies the host.
2. **HTTPS** secures the first transmission.
3. The **Authentication Torus** validates the first key.
4. The **Glass Fortress** activates the directional session.
5. The **Private Torus** takes control.
6. The **Public Torus** provides static geometry.
7. DNS and HTTPS are abandoned.

18.4.1. Full Trust Case

- DNS valid
- HTTPS valid → The first key is transmitted automatically.

18.4.2. Partial Trust Case

- DNS valid
- HTTPS absent → User confirmation required.

18.4.3. Suspicion Case

- DNS suspicious
- HTTPS absent or invalid → Connection blocked until explicit validation.

18.5. Why TorusNet Still Uses DNS and HTTPS

TorusNet is a **post-Internet directional architecture**. However, today's ecosystem still requires:

- domain names,
- TLS certificates,
- HTTP(S) servers.

DNS and HTTPS therefore act as a **bridge** between:

- classical Internet,
- directional Internet.

18.5.1. Final Objective: Total Removal

In future versions:

- DNS will be replaced by **directional DNS**,
- HTTPS will be replaced by **native directional validation**,
- certificates will be replaced by **directional IDs**,
- connections will be fully geometric.

18.6. Interaction with Directional Updates

MAJ 1 — ID Inversion and Mutation

Once the first key is validated:

- the starting case's ID mutates,
- the global ID is created,
- DNS and HTTPS are no longer needed.

MAJ 2 — Removal of Automatic Renewal

The directional session becomes:

- continuous,
- living,
- autonomous.

MAJ 3 — Unique Torus ID

If a key contains the unique torus ID:

- the client detects a directional renewal,
- the session switches to controlled reset mode,
- DNS and HTTPS are not reused.

18.7. Summary of Chapter 18

DNS and HTTPS authentication:

- serves only as an entry airlock,
- protects the first directional key,
- prevents classic attacks,
- validates the requested host,
- disappears once directional geometry activates.

TorusNet:

- uses DNS and HTTPS out of necessity,
- but does not depend on them,
- replaces them immediately with directional mechanics,
- aims to eliminate them entirely in future versions.

This chapter completes TorusNet's entry logic and prepares the transition to **100% directional authentication**.

Conclusion (English Translation)

The development of TorusNet and Mondus marks a profound break with the foundations of classical Internet architecture. By replacing transmissive protocols with a directional architecture rooted in the geometry of toruses and the material identity of machines, the project inaugurates a new way of conceiving digital communication: non-transmissive, non-derivable, non-clonable, and inviolable by design.

The successive updates have progressively established the foundations of an autonomous network capable of operating without traditional cryptography. They have also defined the mechanisms of directional permutation, hardware validation, orbital reading, and structural security that make each session unique, unpredictable, and impossible to reproduce.

Mondus is not an evolution of the Internet: it is a complete alternative, a full reinvention of what a global network can be. By relying on TorusNet as its directional engine, Mondus proposes a post-Internet model where security is no longer an add-on, but a natural property of the structure itself.

Ultimately, whether the project is adopted by institutions or released as open-source, Mondus is intended to become a global standard capable of serving sovereign, industrial, scientific, and civilian needs. It opens the way to a new technological era in which networks no longer transmit data, but directions, and where geometry becomes the foundation of digital security.

Mondus thus represents the transition toward a future in which the network is no longer merely a tool, but a living, directional, autonomous architecture — the first true post-Internet network.

The Architech August 2026